Phase 2 – Mobile Development Completed

Student: Dillan Ycoy

Project: OrderFlow

1. GitHub Link

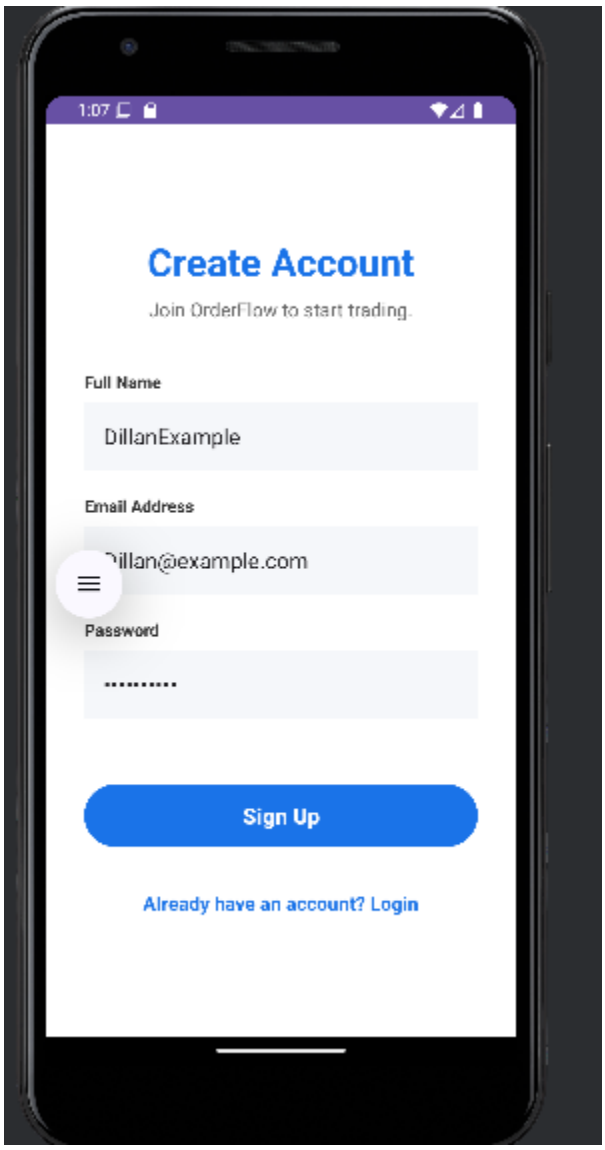
Repository: `https://github.com/dillandeluxe/IT342-Ycoy-OrderFlow.git`

2. Final Commit

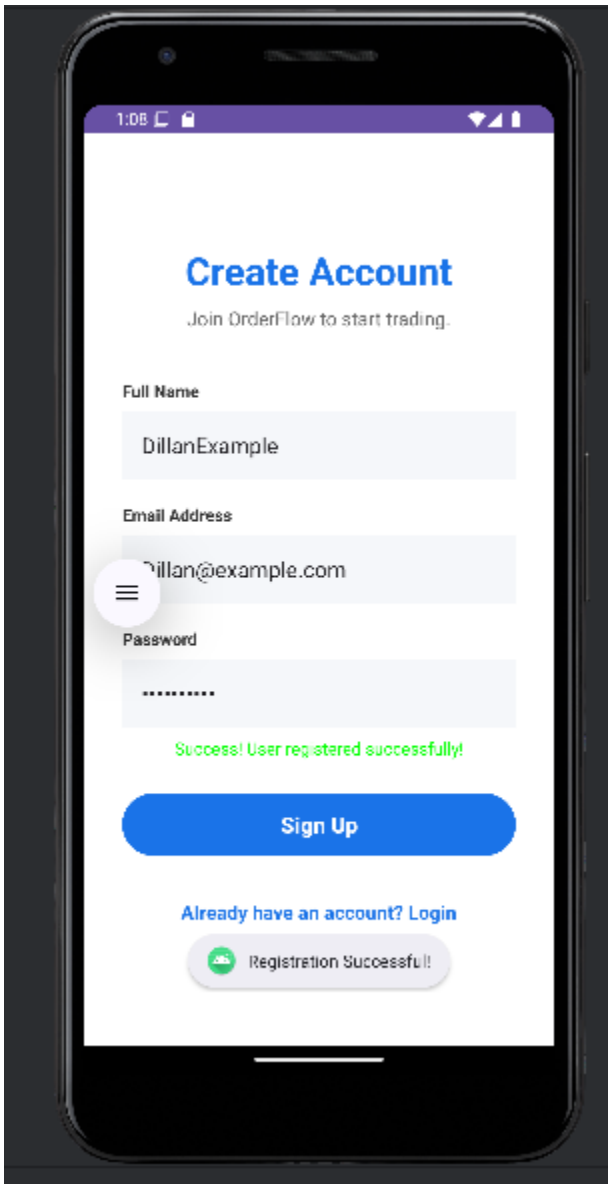
Message: `IT342 Phase 2 – Mobile Development Completed`
Commit Link/Hash:
`https://github.com/dillandeluxe/IT342-Ycoy-OrderFlow/commit/684cfae175d299605017c3dacf7b146dd56b5493`

3. Screenshots

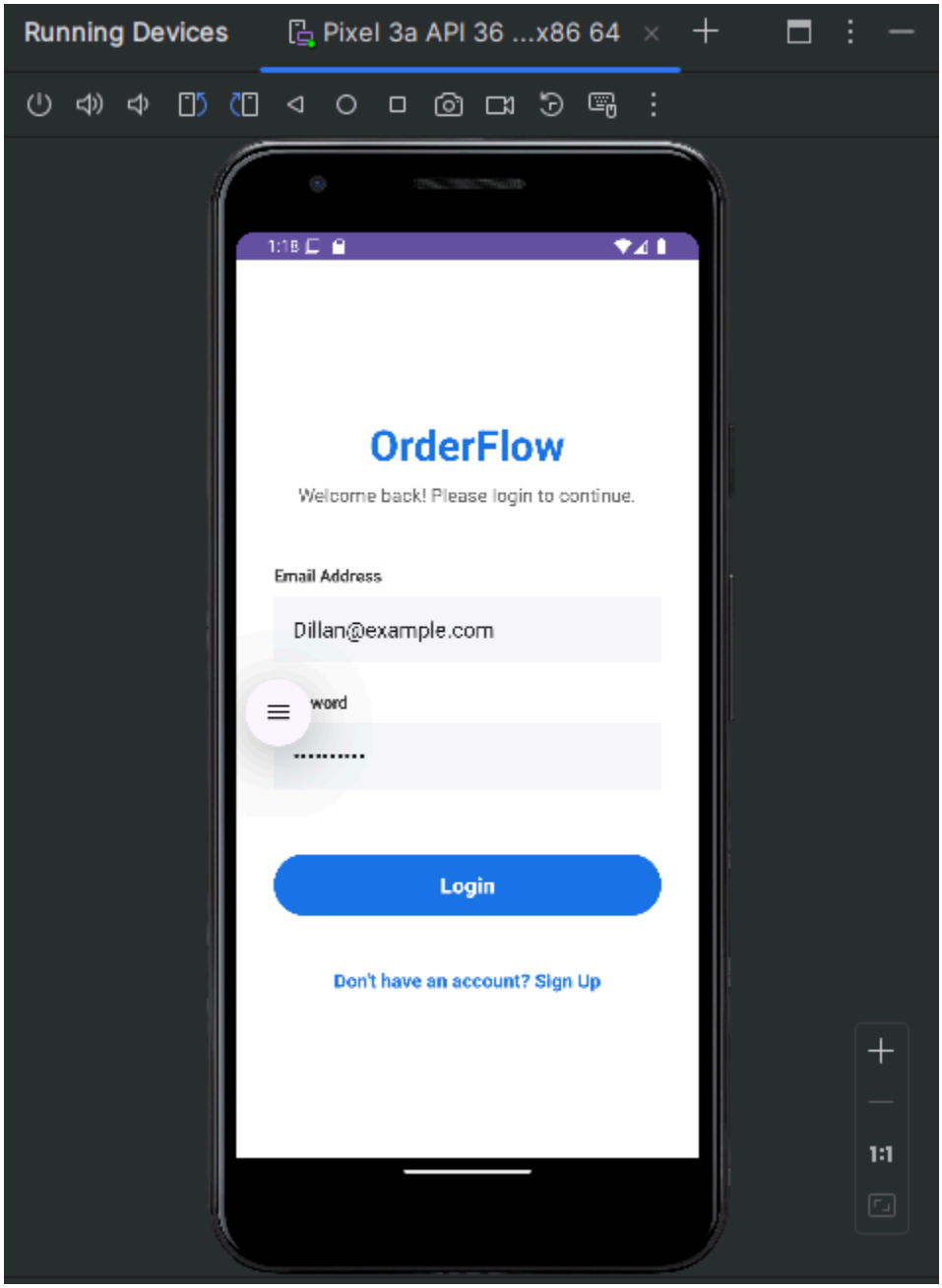
- **Registration screen:**



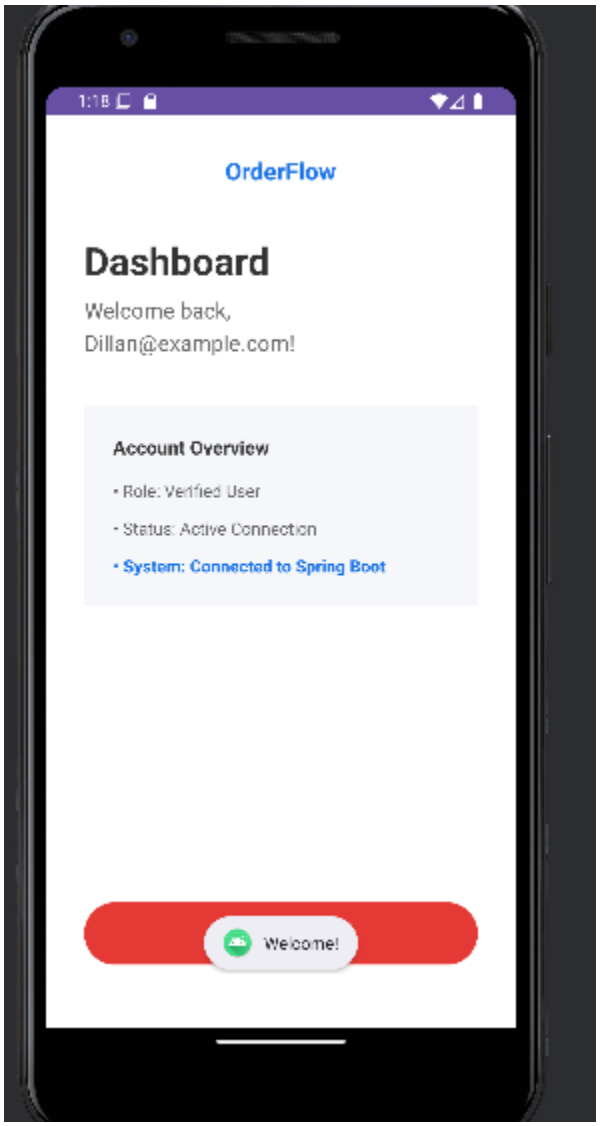
- **Successful registration:**



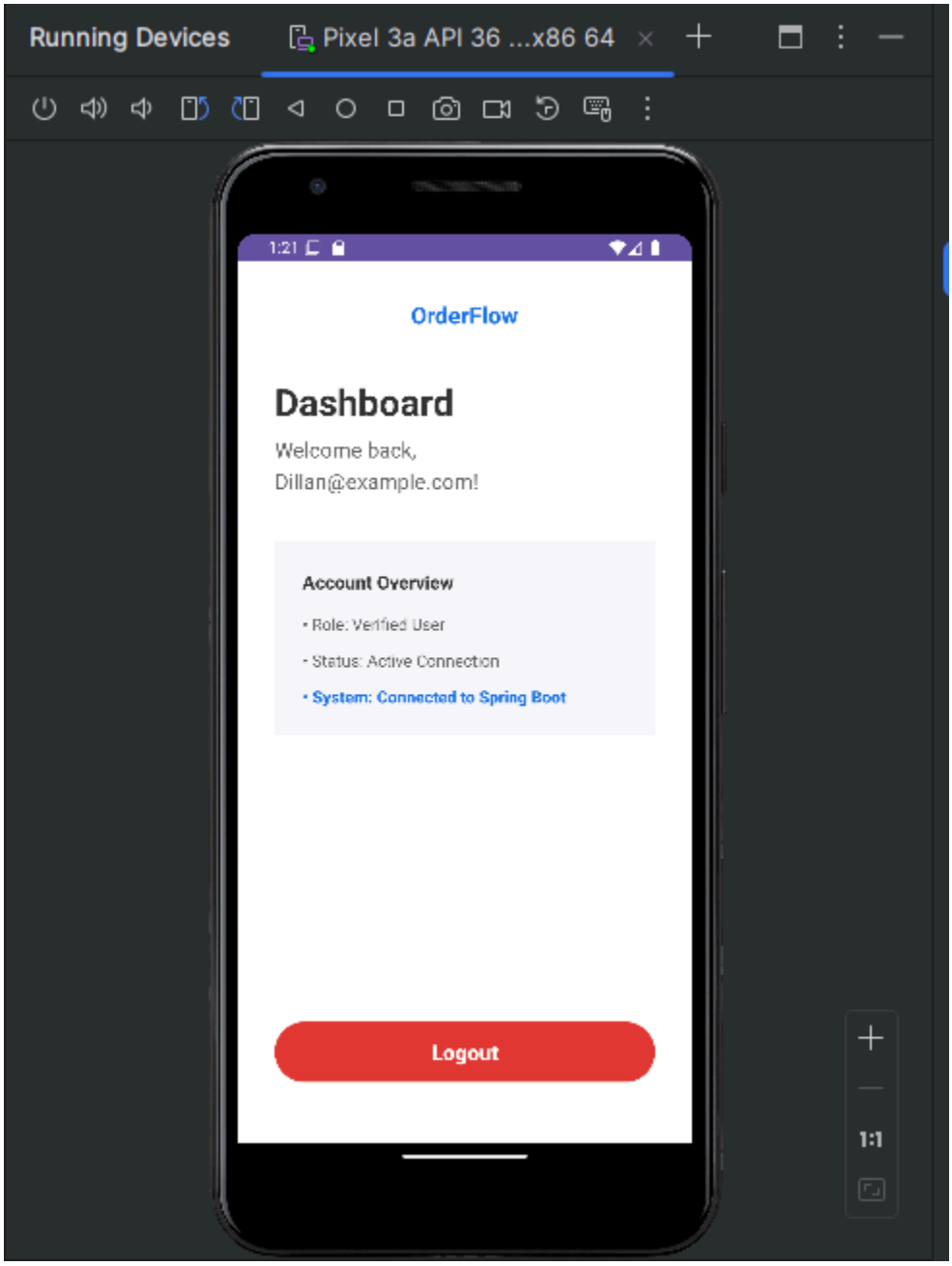
- **Login screen:**



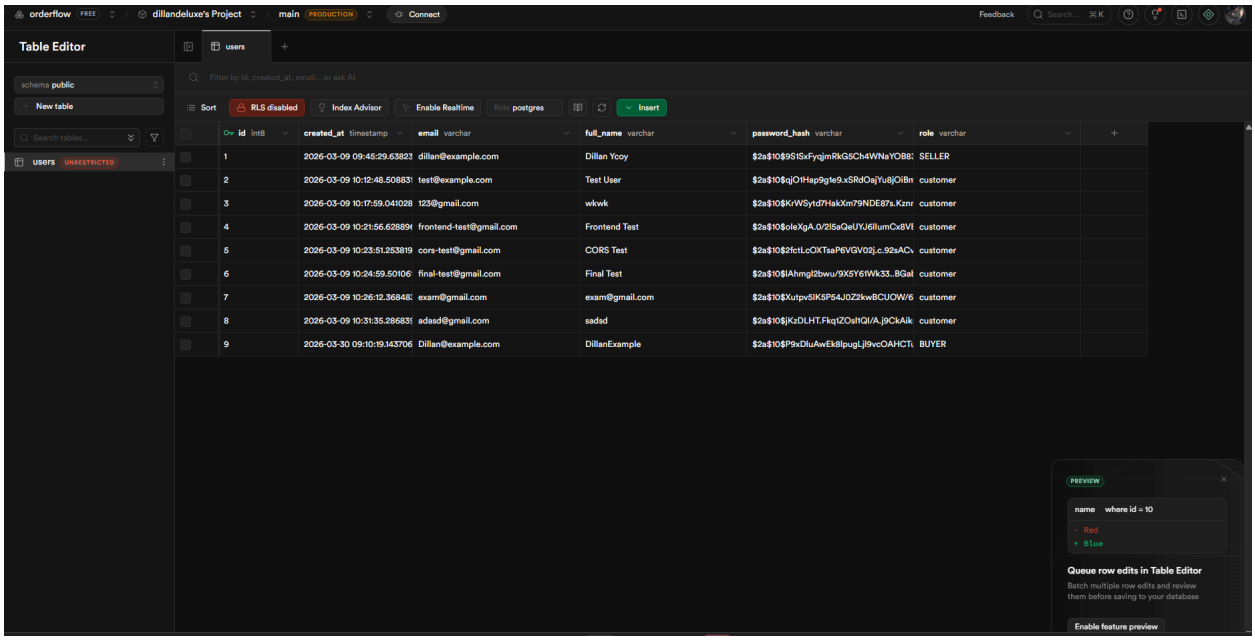
- **Successful login:**



- **After login screen:**



- **Database record:**



---

4. Short Summary

How Registration Works The registration flow begins with a modern XML layout (`activity_register.xml`) that captures the user's Full Name, Email, and Password using standard Android `EditText` views. When the "Sign Up" button is pressed, the Kotlin logic validates that no fields are left empty. **In strict adherence to the project's Software Design Document (SDD), which designates the mobile application exclusively for customers, the system does not prompt for an account type. Instead, the data is packaged into a `RegisterRequest` data class that automatically assigns the role of "BUYER" (reserving "SELLER" functionality for the React Web App).** Using Kotlin Coroutines (`Dispatchers.IO`) to ensure the main UI thread isn't blocked, the app makes an asynchronous network call to the backend. Upon a successful HTTP response, the app switches back to the Main thread (`Dispatchers.Main`) to display a success message and prompt the user to log in.

How Login Works The login process uses a similar architecture. The `LoginActivity` captures the user's credentials and packages them into a `LoginRequest`. The Coroutine sends the payload to the Spring Boot authentication endpoint. If the server validates the credentials and returns a successful response, the app utilizes an Android `Intent` to transition the user to the `DashboardActivity`. The user's email is passed to the next screen using `intent.putExtra()`. Finally, the `finish()` method is called on the Login screen to remove it from the backstack, preventing the user from using the physical "Back" button to bypass authentication.

API Integration Used The mobile frontend communicates with the backend via **Retrofit2**, a type-safe HTTP client for Android. The `ApiClient` object is configured with a base URL pointing to the local Spring Boot server (`http://10.0.2.2:8080` for the Android Emulator). Data serialization and deserialization are handled automatically by the **GsonConverterFactory**, converting Kotlin data classes into the JSON format expected by the Spring Boot REST controllers. The backend itself is connected to a remote **Supabase PostgreSQL** database to permanently persist the user accounts.